

XI: Réseaux de neurones récurrents

Guillaume Guex guillaume.guex@unil.ch

Réseaux de neurones récurrents

Lorsque l'on cherche à modéliser des **séquences** $(\mathbf{x}_1, \dots, \mathbf{x}_I)$, les perceptrons multicouches ont plusieurs problèmes :

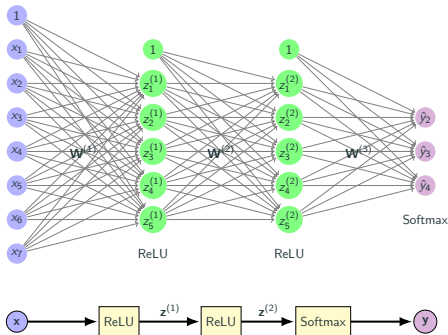
- La notion de **dépendance temporelle**, p.ex. la valeur de $\mathbf{x}_t$ qui dépend certainement de $\mathbf{x}_{t-1}$, n'est pas prise en compte.
- L'apparition d'une sous-séquence sera **traitée différemment selon sa place**, p.ex. « J'ai suivi un cours de ML en 2022. » $\neq$ « En 2022, j'ai suivi un cours de ML. ».
- Les **séquences de taille variable** sont difficiles à traiter.

Les **réseaux de neurones récurrents** permettent de résoudre ces différents problèmes.

Représentation des réseaux de neurones

Ici, nous allons schématiser nos réseaux de cette manière :

(inspiré de <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.)

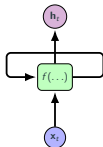


Les **flèches** représentent un transfert d'un vecteur et les **rectangles** une couche de neurone avec activation.

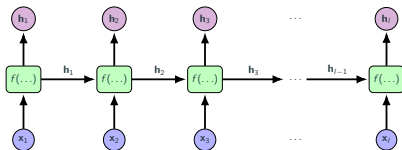
(D'autres notations arriverons par la suite.)

Réseaux de neurones récurrents

Un **réseau de neurones récurrents** se définit par une architecture qui comporte une **connexion récurrente**.



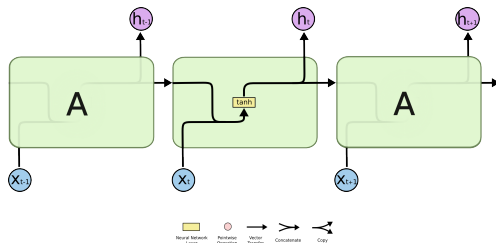
On aura $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$, où $\mathbf{h}_t$ s'appelle l'**état caché** au temps t .
Lorsque l'on passe une séquence $(\mathbf{x}_1, \dots, \mathbf{x}_I)$, on peut **déplier** le réseau :



La fonction $f(\dots)$ représente la **cellule récurrente** du réseau, responsable du **partage des poids**.

Réseaux de neurones récurrents original

Le réseau de neurones récurrent **original** est le suivant :



Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

$$\mathbf{h}_t = \tanh(\mathbf{W}[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b})$$

A l'étape t , le $\mathbf{h}_{t-1}$ est **concaténé** avec l'entrée $\mathbf{x}_t$, puis passe dans une **couche de neurones avec fonction d'activation tanh** pour donner le nouvel état caché et la sortie $\mathbf{h}_t$.

Explosion ou disparition du gradient

Cependant, la récurrence de ce type de réseau amène au **problème d'explosion ou de disparition du gradient** :

- Pour certains poids, les gradients vont devenir **très élevés**, ce qui amène à des pas d'entraînement trop grands. On peut résoudre ce problème en procédant au **gradient clipping**.
- Plus grave, pour certains poids, les gradients vont **disparaître**, ce qui fait que les poids ne sont jamais corrigés (le réseau « oublie » rapidement).

Ces phénomènes se produisent car les poids sont réutilisés de **manière multiplicative** plusieurs fois dans la séquence.

Long Short-Term Memory (LSTM) et Gated Recurrent Unit (GRU)

Long short-term memory

Les réseaux **long short-term memory (LSTM)**, conceptualisés en 1996 par S. Hochreiter et J. Schmidhuber, permettent de résoudre le problème de la disparition du gradient.

La cellule récurrente possède deux types de mémoire :

- La **mémoire à court terme**, représentée par l'état caché $\mathbf{h}_t$.
- La **mémoire à long terme**, représentée par l'**état de la cellule**, noté $\mathbf{c}_t$.

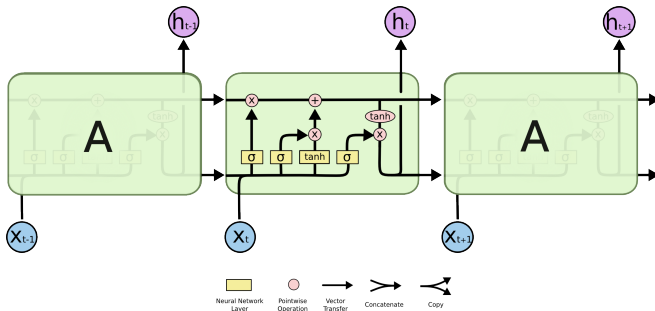
On aura les relations récurrentes suivantes :

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{c}_{t-1}, \mathbf{x}_t) \quad \mathbf{c}_t = g(\mathbf{h}_{t-1}, \mathbf{c}_{t-1}, \mathbf{x}_t)$$

L'état caché $\mathbf{h}_t$ constitue également la **sortie** au temps t .

Long short-term memory

La cellule se présente sous la forme suivante :

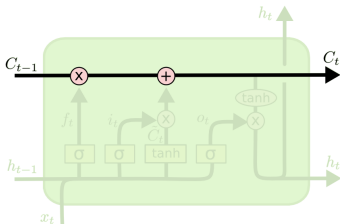


Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Elle est constituée de trois **portes** différentes : **forget**, **input** et **output**.

La mémoire à long terme

En haut de la cellule, on trouve la **mémoire à long terme** c_t :

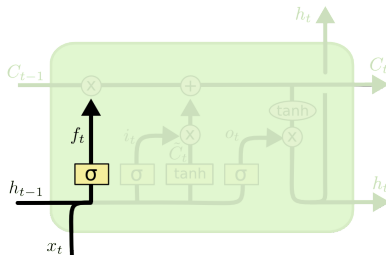


Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Elle ne subit que peu de modifications : une **multiplication de ses composantes** par des valeurs dans $[0, 1]$ (forget) et une **addition à ses composantes** de valeurs dans $[-1, 1]$ (input).

La porte forget

Le résultat de la **porte forget**, f_t , est la sortie d'une couche de neurones avec **sigmoïde** du vecteur concaténé h_{t-1} et x_t .



Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

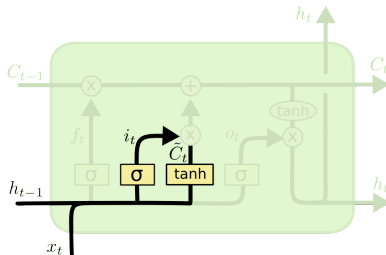
$$f_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

Cette porte définit, en fonction de l'entrée et de l'état caché, ce qu'il faut **oublier** dans la mémoire à long terme.

La porte input

La **porte input** est composée de deux éléments :

- le **candidat de mémoire à long terme** $\tilde{c}_t$, avec composantes dans $[-1, 1]$.
- l'**importance du candidat** i_t , avec composantes dans $[0, 1]$.

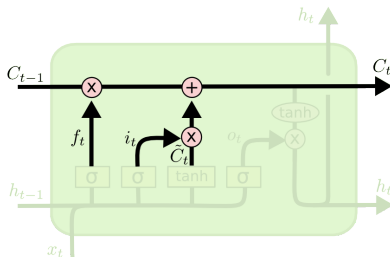


Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

$$\tilde{c}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

Mise à jour de la mémoire à long terme

La mémoire à long terme est **mise à jour** avec ces deux composantes :



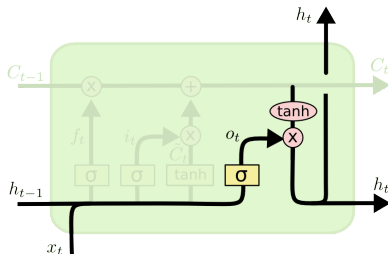
Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

On oublie une partie de l'ancienne mémoire, et on ajoute une partie du candidat.

La porte output

Finalement, la **porte output** va sélectionner, parmi les éléments de la nouvelle mémoire à long terme c_t (après une $\tanh$), l'**état caché suivant** h_t .



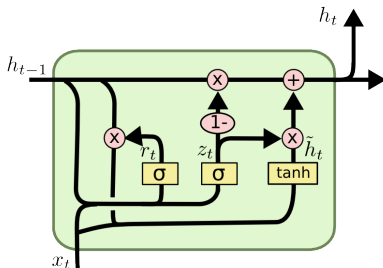
Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

$$h_t = o_t \odot \tanh(c_t) \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Cet état caché sert de **sortie et de mémoire à court terme**.

Gated Recurrent Unit (GRU)

En 2014, K.Cho et al. ont développé une nouvelle cellule récurrente, plus simple et avec performances presque équivalentes, appelée **Gated Recurrent Unit (GRU)**.

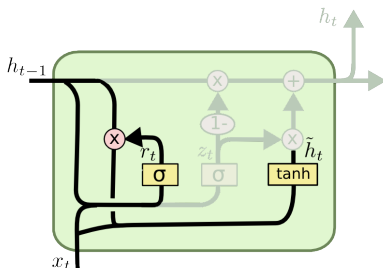


Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Elle ne possède qu'un seul type de mémoire h_t , et deux portes : **reset** et **update**.

La porte reset

La **porte reset** va annuler une partie de l'état caché grâce à $\mathbf{r}_t$, et va créer un candidat au nouvel état caché : $\tilde{\mathbf{h}}_t$.

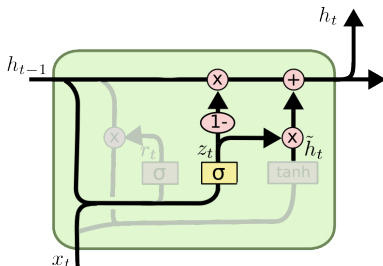


Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \quad \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$$

La porte update

La **porte update** va créer un nouvel état caché $\mathbf{h}_t$ en faisant un mélange, déterminé par $\mathbf{z}_t$, entre l'ancien état caché et le candidat.



Source : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$$

Réseaux récurrents en pratique

Bien qu'un réseau de neurones récurrent peut se déplier pour traiter des séquences de taille variable, généralement, les **batches contiennent des séquences d'une taille fixe**.

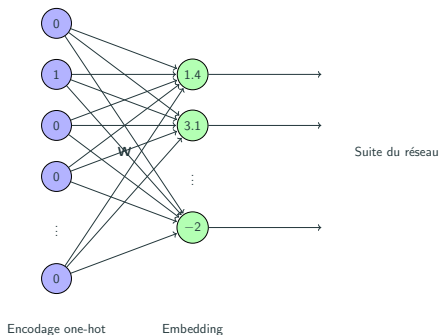
Pour cela, on va compléter les séquences plus courtes que les autres avec un signe désignant un **padding**.

« J'apprends le machine learning » « C'est sympa »
→
« J'apprends le machine learning » « C'est sympa [PAD] [PAD] »

On peut aussi éventuellement l'ajouter à la fin de toutes les phrases pour que le modèle apprenne à conclure.

La couche d'embedding

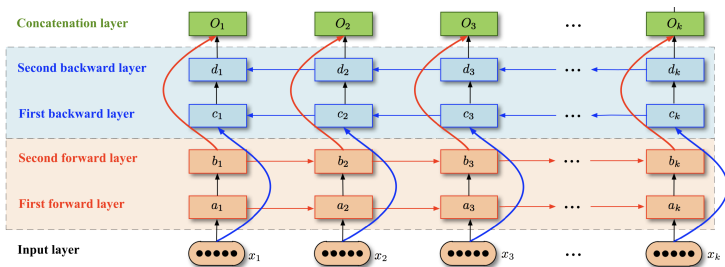
Lorsque l'on traite une **séquence de variables catégorielles**, l'encodage one-hot de ces dernières peut être transformé en vecteur (de dimensionalité réduite) grâce à une **couche d'embedding**.



L'**embedding** va être entraîné pour représenter au mieux les variables selon la tâche.

Réseaux multicouches et bidirectionnels

Les réseaux récurrents possèdent souvent **plusieurs couches** et peuvent être **bidirectionnels** :



Source : J. Zhou et al. (2019) *Sentiment Analysis of Chinese Microblog Based on Stacked Bidirectional LSTM*

- Ajouter des couches **augmente le pouvoir d'abstraction**.
- Rendre le réseau bidirectionnel **permet de prendre en compte les deux côtés de la séquence**.